

TravelOn 中期检查报告

项目名称：TravelOn——微服务驱动的一站式旅游平台

小组：26NULLptr

中期检查日期：2026-08-29

报告更新日期：2026-08-31

复核基线：main 分支 42977db

仓库：[QzlabQ/travel-on-2026NULLptr](#)

文档用途：中期检查单 PDF 提交版

快速检查导航

检查人员可从下列入口直接跳转到对应证据：

- [中期检查要求与完成结论](#)
- [原系统基线与 Git 标签](#)
- [8 个业务场景、测试记录与界面证据](#)
- [需求、系统级图、组件级图、对象级图与追溯表](#)
- [前端、后端与数据库容器](#)
- [GitHub Actions 自动构建与测试](#)
- [微服务划分、服务接口与数据表归属](#)
- [扩展成果：K3s 持续部署](#)
- [全部在线提交材料](#)

摘要

TravelOn 是一个覆盖“行前规划—预订支付—行程管理—行后分享”的一站式旅游平台。项目采用 React 前端、Spring Boot/Spring Cloud 微服务、Python AI Agent，以及 PostgreSQL、MongoDB、RabbitMQ 等数据与消息基础设施，并通过 Docker Compose 提供本地容器化运行环境。

截至本报告复核基线，项目已完成 8 个确认业务场景的需求说明、系统级模型、组件级模型、对象级模型及端到端追溯；前端、后端、数据库和中间件已有容器化方案与验证记录；GitHub Actions CI 已能在 Pull Request 和 main 分支变更时自动执行构建、单元测试、集成测试及报告上传；微服务划分图、服务接口清单与数据表归属方案均已形成。项目还在中期要求之外增加了面向“256”单节点 K3s 环境的 CD 流程。

本报告采用“单一主报告 + 在线详细材料”的提交方式。PDF 正文给出检查结论、架构说明、场景覆盖和关键证据，详细需求、设计、接口与测试文档通过可点击链接保留，避免把多个 Markdown 直接拼接成重复且难以阅读的超长 PDF。

1. 中期检查要求与完成结论

中期检查要求	状态	完成情况与主要证据
--------	----	-----------

中期检查要求	状态	完成情况与主要证据
原系统能够启动；确认后的业务场景全部可运行；已打 Git 标签	已完成	重构前基线已使用 <code>monolith-start</code> 标签保存，指向 <code>4d3f470</code> ；BS-01 ~ BS-08 均有实现、测试记录和业务截图。证据入口： 版本标签 、 业务场景与运行证据 。
所有业务场景的需求、系统级图、组件级图、对象级图和追溯表完成	已完成	8 个业务场景均已建立“需求—用例—系统级模型—组件级模型—对象级模型—代码—测试”链路。证据入口： 需求 、 三层设计与追溯 。
前端、后端、数据库容器能够启动	已完成	React/Nginx 前端、Java/Python 后端、PostgreSQL、MongoDB 与 RabbitMQ 均有 Docker 配置；部署测试 <code>TC001</code> ~ <code>TC008</code> 已记录通过。证据入口： 容器化部署与运行 。
CI 能够自动完成构建和测试	已完成	GitHub Actions CI 在 PR、 <code>main</code> push 和手动触发时执行编译、前端 Docker 构建、单元测试、集成测试和报告上传。证据入口： GitHub Actions CI 。
微服务划分图、服务接口清单和数据表归属方案完成	已完成	三项材料均已合并至 <code>main</code> ，并分别说明目标服务边界、REST/WebSocket/RabbitMQ 接口和数据所有权。证据入口： 微服务 、 接口与数据归属 。

结论：课程列出的中期检查项目均已有仓库证据。正式提交 PDF 前，建议再补入最新 GitHub Actions、容器运行状态和 K3s 部署成功页面截图，使“文档证据”和“现场证据”同时具备。

2. 项目范围与系统架构

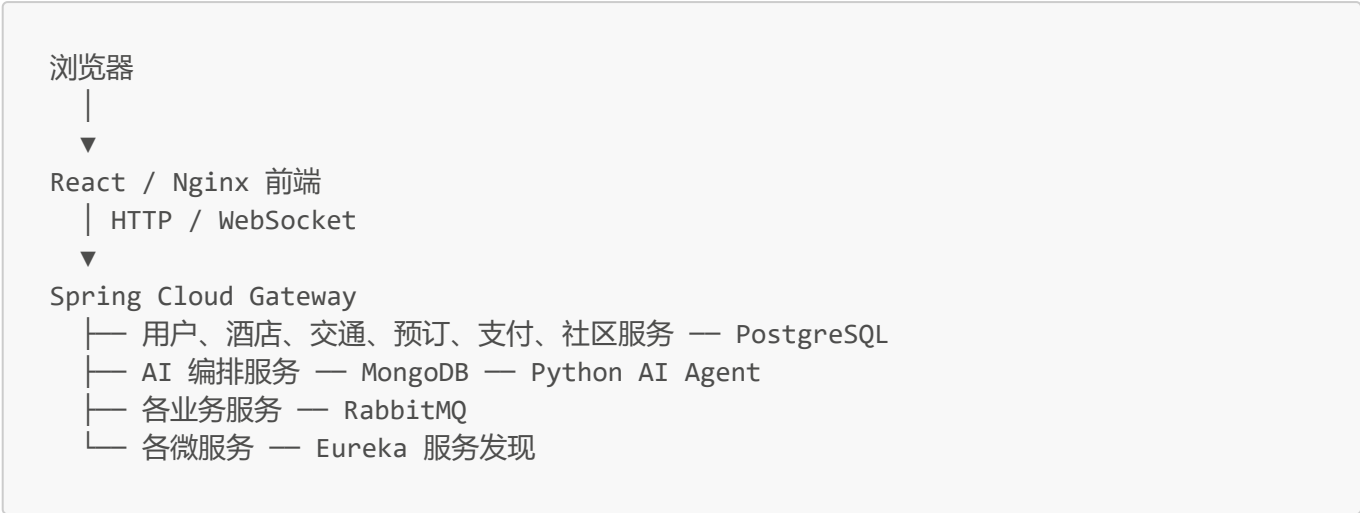
2.1 项目目标

项目目标是在一个 Web 应用中提供账户与常用旅客维护、酒店/机票/火车票查询预订、统一订单、模拟支付与退款、旅行时间线、社区互动和 AI 行程规划。当前版本以课程演示环境下的业务闭环为重点，不以真实供应商接入和大规模高并发交易为验收目标。

2.2 技术架构

层次	主要技术与职责
用户界面	React、TypeScript、MUI；展示查询、预订、订单、社区与 AI 规划页面
统一入口	Spring Cloud Gateway；统一 REST/WebSocket 入口和路由
服务发现	Eureka Discovery；维护微服务实例注册信息
业务服务	用户、酒店、交通、预订、支付、社区、AI 编排等 Spring Boot 服务
AI 能力	Python Agent 与 Java AI 编排服务共同完成会话、行程和快照管理
数据层	PostgreSQL 保存业务数据，MongoDB 保存 AI 会话与快照
异步协作	RabbitMQ 处理预订、库存和服务间异步消息
交付运行	Docker Compose 用于开发/集成环境；K3s 用于“256”部署环境

系统调用关系可以概括为：



完整的现状与目标架构图见[微服务划分图](#)和[代码核实基线](#)。

3. 原系统基线与版本管理

仓库通过注释标签保存重构前原系统基线，便于复现、对比和回退：

项目	内容
标签	monolith-start
标签说明	version before refactoring
指向提交	4d3f470db5fb9a8d4366950fe42943f90bb69584
标签建立时间	2026-08-24
本报告复核基线	42977db7d3d7736e08f80bfb1bded3ef24ab4879
复核基线日期	2026-08-31

标签的意义不是复制一份代码，而是给确定的 Git 提交建立不可混淆的里程碑名称。教师或组员可以使用以下命令核验：

```
git show --no-patch --format=fuller monolith-start
git rev-parse monolith-start^{ }
git log --oneline --decorate main
```

建议在最终提交时再为中期交付建立一个明确标签，例如 `midterm-2026-08-31`。该标签应由项目负责人在确认最终提交 SHA 后创建，避免标签先于材料定稿。

4. 业务场景及运行证据

4.1 确认后的业务场景

编号	业务场景	角色与入口	可验收结果	状态
BS-01	账户初始化与出行人维护	游客/注册用户，登录与账户中心	注册或登录后维护资料与常用旅客	通过
BS-02	酒店查询与住宿预订	注册用户，酒店页面	按条件查询酒店、选择房型并创建待支付订单	通过
BS-03	机票查询与机票预订	注册用户，机票页面	查询航班和舱位并创建待支付订单	通过
BS-04	火车票查询与火车票预订	注册用户，火车票页面	查询车次、选择席别并创建待支付订单	通过
BS-05	订单支付与售后处理	注册用户，订单详情页	模拟支付、取消、退款及状态流转	通过
BS-06	统一订单与旅行时间线管理	注册用户，订单中心	查询订单、流水、退款和有效行程	通过
BS-07	社区内容分享与互动	游客/注册用户，社区页面	浏览或执行发布、评论、点赞、收藏和评价	通过
BS-08	AI 行程规划与版本管理	注册用户，AI 规划页面	创建规划，查看、编辑、比较和恢复快照	通过

各场景的前置条件、主成功流程、异常流程、业务规则和完成结果见[业务场景说明](#)。

4.2 测试与可复现证据

- 功能、部署、边界与异常用例编号覆盖 TC001 ~ TC605，详见[测试文档](#)。
- 2026-08-27 的统一单元测试记录为 155/155 通过、0 失败、0 错误、0 跳过，详见[本地测试与覆盖率结果](#)。
- 统一测试入口由 mise.toml 和 tests/run_tests.py 提供，可运行单元、集成、Playwright E2E 和完整测试。
- 任一测试模块失败时，统一入口返回非零退出码并生成日志和结构化汇总，可直接供 CI 判断成功或失败。

4.3 业务界面证据

以下截图已经保存在仓库中，Markdown 中可直接预览，导出 PDF 时也应作为关键页面证据保留。

图 1 平台首页



图 2 账户与常用旅客

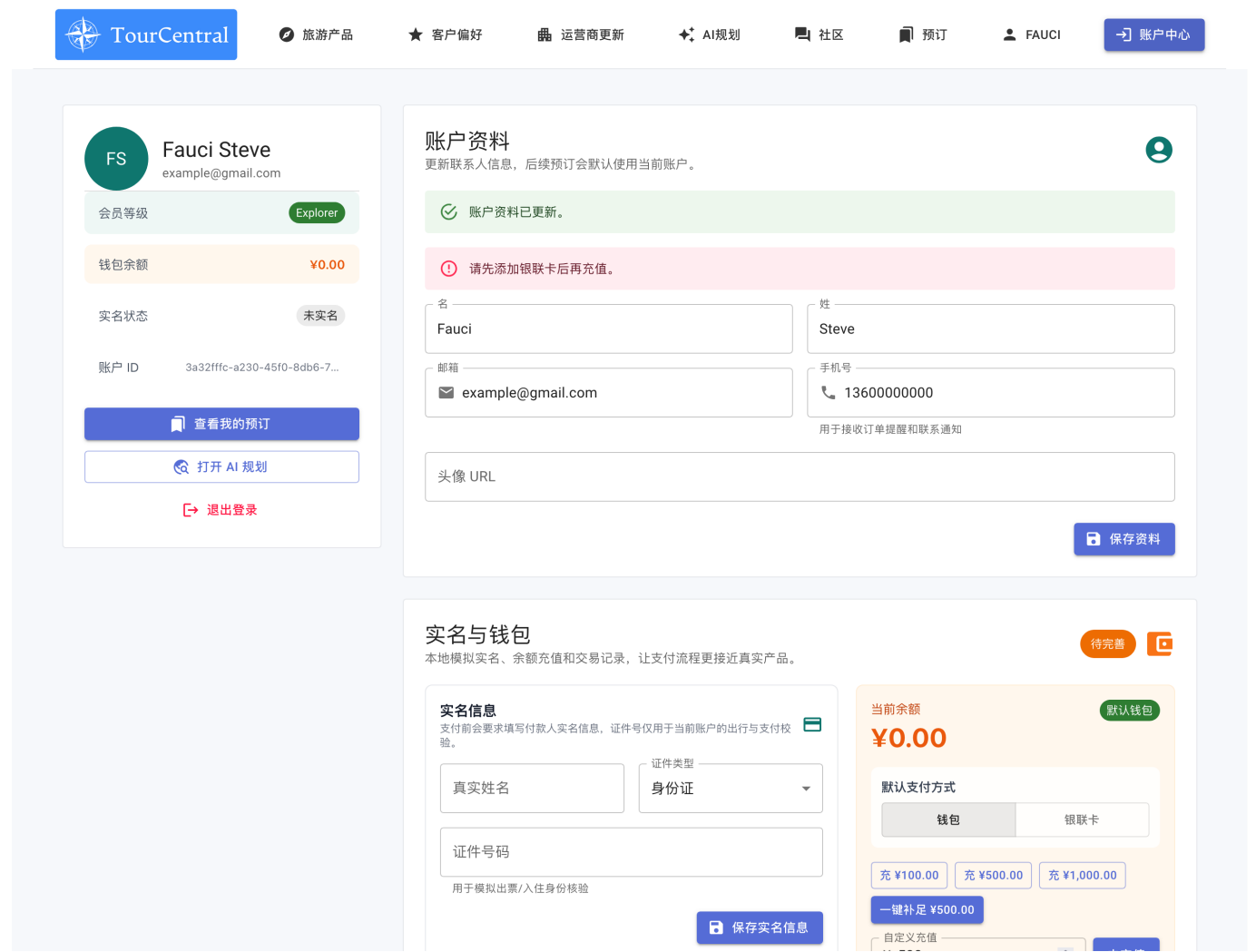


图 3 酒店查询与预订

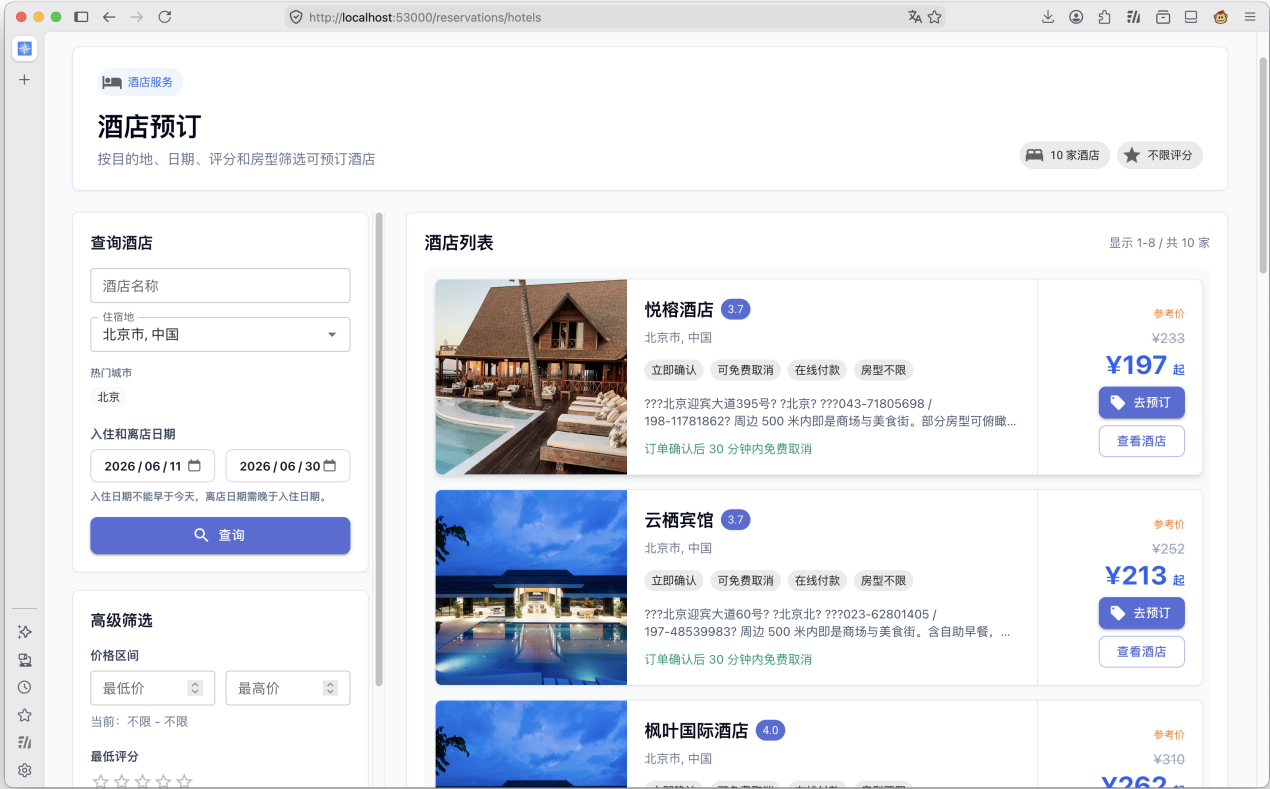


图 4 统一订单列表

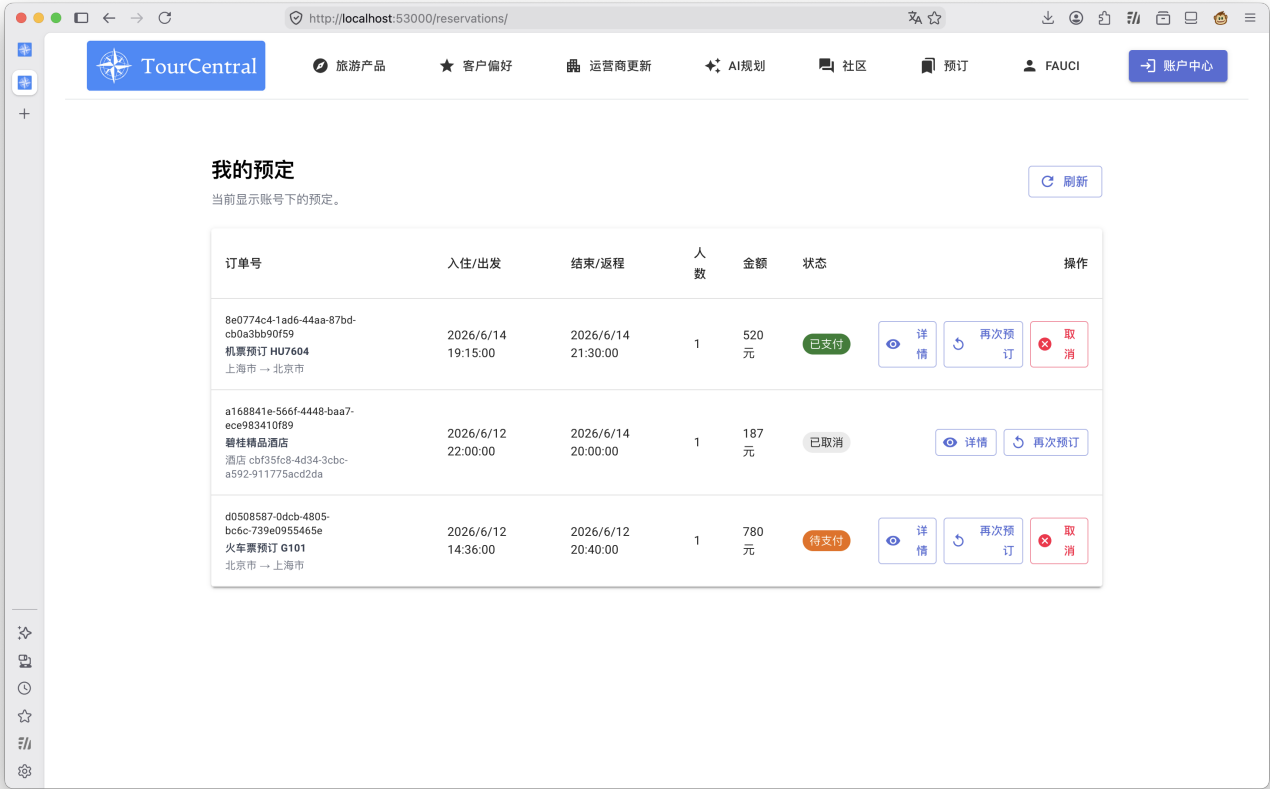


图 5 社区内容与互动

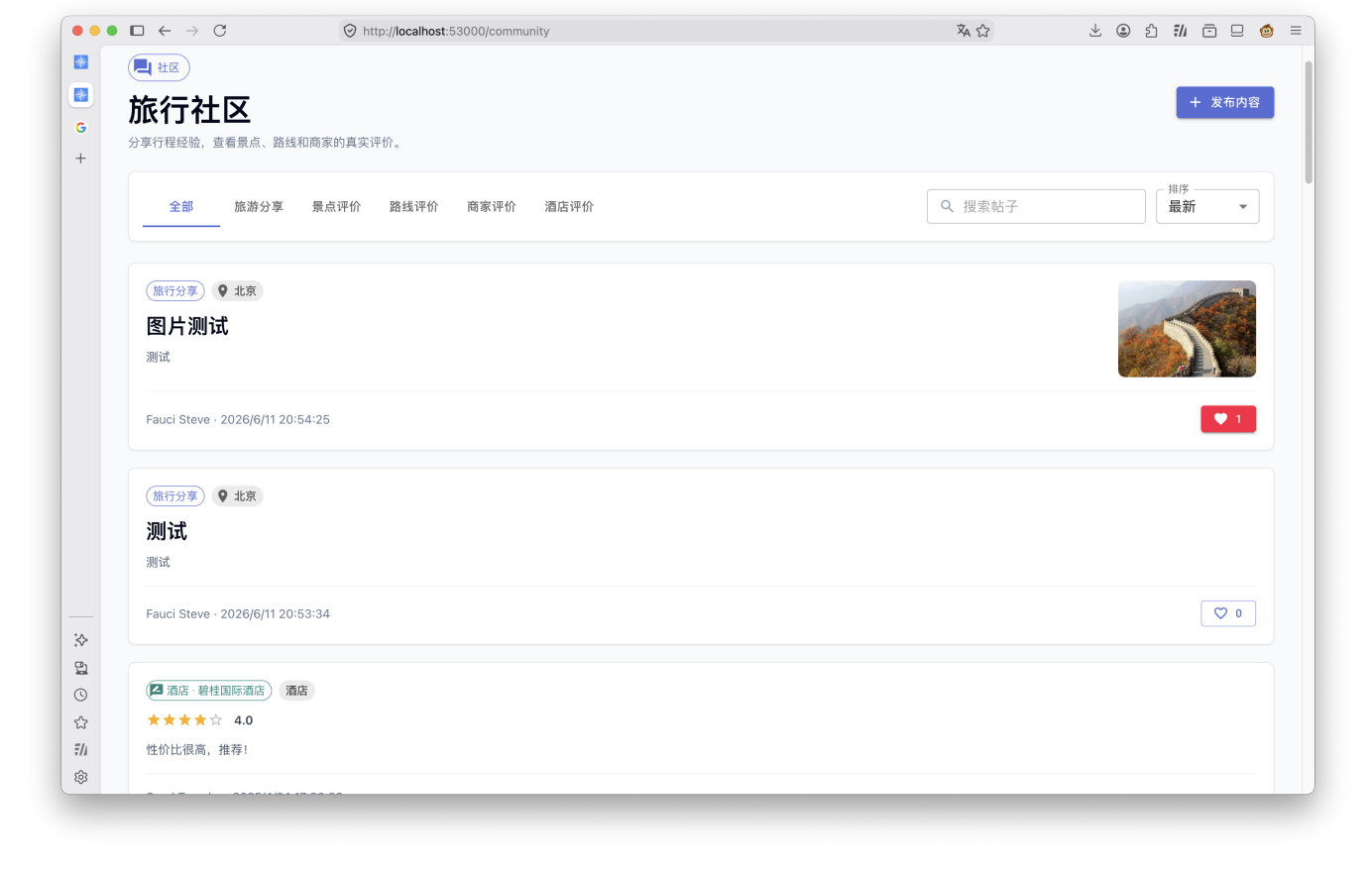


图 6 AI 行程规划结果



其余页面证据包括：注册、登录、火车票、机票、订单详情、支付和AI 规划会话。

5. 需求、三层设计与追溯

5.1 文档分层与职责

层次	交付物	主要内容
需求与系统级	需求规格	系统边界、功能/接口/数据/质量要求，以及各场景的系统级顺序图、状态图或活动图
业务用例	业务场景	角色、前置条件、主流程、异常流程、规则、完成结果和入口
组件级	概要设计	总体架构、服务职责、服务间通信及八个场景的组件协作图
对象级	详细设计	领域对象、关键类、接口处理及八个场景的对象级交互图
端到端追溯	需求追溯表	需求 → 用例 → 三层模型 → 代码 → 测试的双向关系

5.2 八个场景的设计覆盖

场景	系统级模型	组件级模型	对象级模型	代码与测试追溯
BS-01	需求规格 §3.25	BS-01 概要设计 §4.2	详细设计 §5.2	已建立
BS-02	需求规格 §3.25	BS-02 概要设计 §4.3	详细设计 §5.3	已建立
BS-03	需求规格 §3.25	BS-03 概要设计 §4.4	详细设计 §5.4	已建立
BS-04	需求规格 §3.25	BS-04 概要设计 §4.5	详细设计 §5.5	已建立
BS-05	需求规格 §3.25	BS-05 概要设计 §4.6	详细设计 §5.6	已建立
BS-06	需求规格 §3.25	BS-06 概要设计 §4.7	详细设计 §5.7	已建立
BS-07	需求规格 §3.25	BS-07 概要设计 §4.8	详细设计 §5.8	已建立
BS-08	需求规格 §3.25	BS-08 概要设计 §4.9	详细设计 §5.9	已建立

三层模型解决不同问题：系统级图说明用户和外部系统如何参与；组件级图说明哪些服务协同完成场景；对象级图说明服务内部关键对象的职责和交互。追溯表把三层设计进一步连接到源码路径和测试编号，使变更时可以向反向判断影响范围。

5.3 PDF 中图表的处理

详细设计文档中的图主要采用 Mermaid。GitHub 能直接渲染这些图，但普通 Markdown 转 PDF 工具不一定支持 Mermaid。因此本 PDF 主报告只嵌入关键业务截图，并通过在线链接保留完整三层图；若教师明确要求所有模型图离线可见，应先将 Mermaid 批量导出为 PNG/SVG，再合并到最终 PDF，而不能让 PDF 中只出现 Mermaid 源码。

6. 容器化部署与运行

6.1 容器范围

类别	组件	运行方式
前端	React 静态资源、Nginx	travel-ui/Dockerfile

类别	组件	运行方式
后端入口	API Gateway、Eureka Discovery	travel-api/docker-compose.yml
业务服务	hotel、transport、reservation、payment、user、community、AI arrange/agent 等	Docker Compose 编排
数据与中间件	PostgreSQL、MongoDB、RabbitMQ	Docker Compose 编排并配置健康检查/持久化

本地开发与集成环境的典型启动方式为：

```
cd travel-api
docker compose up -d --build
docker compose ps
```

前端可以独立构建运行，也可由后续 K3s 清单统一部署：

```
docker build -t travel-ui:local travel-ui
docker run -d --name travel-ui -p 80:80 travel-ui:local
```

完整环境变量、数据库初始化、端口、服务发现与故障排查方式见[部署文档](#)。

6.2 已有验证记录

部署测试 TC001 ~ TC008 已记录以下结果：Compose 构建启动通过；PostgreSQL 健康检查和初始化通过；JPA schema 校验通过；Gateway 与业务服务可注册到 Eureka；网关路由和前端首页可访问。正式 PDF 最好补一张最新 docker compose ps 或 K3s kubectl get pods 截图，以直观证明提交时的运行状态。

7. GitHub Actions 持续集成（CI）

7.1 触发方式

CI 工作流有三种触发方式：

- 向 main 创建或更新 Pull Request；
- push 到 main；
- 在 GitHub Actions 页面手动触发。

CI 使用 GitHub 托管的 Ubuntu Runner，执行超时上限为 90 分钟；同一分支出现更新时取消过时运行，避免浪费资源。

7.2 自动构建与测试流程

```
检出代码
→ mise 安装固定版本的 JDK / Node.js / Python 与项目依赖
```

- 编译打包全部 Java 服务
- 检查 Python Agent 源码
- 构建 React 前端
- 构建前端 Docker 镜像
- 执行 Java + Python + 前端单元测试
- 启动集成环境并运行集成测试
- 上传测试报告（保留 14 天）

CI 的核心价值是把“在某位成员电脑上能运行”变成“每个 PR 都由一致环境自动验证”。任何构建或测试步骤失败， workflow 整体失败， PR 页面会显示红色检查结果，从而阻止未经验证的代码被误认为可交付。

已核实的历史成功运行示例：

- [前端 Docker 修复：GitHub Actions 运行记录](#)
- [AI 规划可靠性修改：GitHub Actions 运行记录](#)

运行页面链接在 2026-08-31 复核时可访问。最终提交时仍应以最终 `main` 提交对应的 CI 结果为准，并建议将该绿色运行页面截图放入 PDF。

8. 微服务划分、接口与数据归属

8.1 微服务划分

[微服务划分图](#)同时描述当前运行基线、目标边界和关键调用链。划分遵循以下原则：

- 保持 `/hotels/**`、`/transports/**`、`/reservations/**`、`/users/**`、`/community/**`、`/ai-arrange/**` 等对外路径稳定；
- 酒店和交通能力归入 `travel-core` 业务边界；
- 订单和支付归入 `order` 业务边界；
- 用户、社区与 AI 规划因职责和数据相对独立而保留独立边界；
- Gateway 负责统一入口，Eureka 负责发现，RabbitMQ 负责异步协作。

该方案区分“当前已运行的物理服务”和“后续整合后的目标边界”，避免把设计目标误写成已经完成的代码事实。

8.2 服务接口清单

[服务接口清单](#)按源码记录：

- Gateway 对外 REST 路径、方法和主要参数；
- AI 与订单相关 WebSocket；
- RabbitMQ queue、exchange 和 routing key；
- 鉴权方式、兼容策略及服务合并前后差异；
- 接口变更对前端和其他服务的影响。

接口清单使服务边界不只停留在架构图上，而是可以落实到可调用、可测试的契约。

8.3 数据表归属方案

[数据表归属方案](#)明确每类数据只能由归属服务直接管理：

目标服务	数据库	主要数据
------	-----	------

目标服务	数据库	主要数据
travel-core-service	travel_core_db	城市、酒店、房型、房间预订、票务模板与查询视图
order-service	reservation_db	订单、旅客快照、支付流水、退款记录
user-service	user_db	用户与常用旅客
community-service	community_db	帖子、评论、点赞、收藏、评价、景点和路线
ai-arrange-service	MongoDB ai-arrange-db	规划会话、消息、快照和版本

目标方案禁止服务跨库直接查询和建立跨服务数据库外键。需要其他服务的数据时，通过 HTTP 或 RabbitMQ 协作，从而保持数据所有权和服务自治。

9. 扩展成果：K3s 持续部署（CD）

CD 不是本次中期检查原文中的必选项，但项目已经形成第一版面向“256”环境的自动部署链路。这里的“256”是项目对部署服务器/环境的命名，不是 GitHub 或 Kubernetes 的固定术语。

9.1 部署触发与目标

CD 工作流在 main 的 CI 成功后触发，也支持手动执行。部署 Job 只能被带有 travelon-256 标签的 Linux x64 自托管 Runner 接收，并绑定 GitHub Environment production-256。

main 的 CI 成功

→ 256 自托管 Runner 检出同一个已测试 SHA

→ 按 ops/cd/images.tsv 构建带 sha-xxxxxxx 标签的镜像

→ 将镜像导入本机 K3s containerd

→ 渲染并应用 k8s/base

→ 等待 StatefulSet / Deployment rollout

→ 输出 Pod 状态并执行基础烟雾检查

9.2 安全与可维护性设计

- 使用提交 SHA 作为镜像标签，使部署版本可以追溯到源码；
- 构建清单集中在 ops/cd/images.tsv，新增服务时不必修改高权限 wrapper；
- Runner 通过受限 sudo wrapper 执行 Docker/K3s 操作，并校验调用账号、工作区、标签格式和文件路径；
- CD 会拒绝 privileged、hostPath、hostNetwork、hostPID 和 hostIPC 等高风险清单；
- Kubernetes Secret 由管理员预置，Runner 不保存或写入真实业务密钥；
- 部署前等待各 StatefulSet 和 Deployment 完成 rollout。

9.3 当前边界

当前为单节点 K3s、节点本地构建和导入镜像的方案，适合课程环境和第一轮部署验证。它尚不等同于多节点生产平台：没有完整的自动回滚策略；烟雾检查仍较轻；Runner、构建与业务负载共享同一服务器资源；MongoDB/RabbitMQ 等基础镜像需要事先具备。后续可增加镜像仓库、外部入口探测、自动回滚和备份恢复演练。

10. 提交材料索引

以下链接均指向 GitHub `main` 分支，导出 PDF 后仍可点击；提交时应确保 PDF 阅读器保留超链接。

材料	在线链接	用途
项目说明	README	项目简介、技术栈与启动入口
业务场景	业务场景.md	BS-01 ~ BS-08 用例说明
软件需求规格	需求规格.md	需求与系统级图
概要设计	概要设计.md	组件级设计
详细设计	详细设计.md	对象级设计
需求追溯	需求追溯表.md	需求到测试的端到端追溯
部署说明	部署文档.md	Compose 启动与排障
测试用例	测试文档.md	功能、部署、异常用例与结果
测试结果	testing-results-2026-08-27.md	155 个单元测试及覆盖率
CI 配置	ci.yml	自动构建与测试
CD 配置	cd.yml	CI 成功后的 K3s 部署
微服务划分	architecture-diagram.md	当前架构、目标边界与调用链
服务接口清单	service-interface-inventory.md	REST、WebSocket、RabbitMQ 契约
数据表归属	database-ownership-plan.md	数据所有权与迁移方案
K3s 清单	k8s/base	256 环境 Kubernetes 资源

11. 总结

TravelOn 已从单纯的功能实现推进到具有明确需求、分层设计、端到端追溯、自动测试、容器化运行和持续交付雏形的工程化项目。八个确认业务场景均有对应的需求、系统级/组件级/对象级模型、源码位置和测试证据；微服务边界、接口契约和数据归属也已形成可执行方案。GitHub Actions CI 解决了自动构建与测试问题，K3s CD 则进一步验证了从已测试提交到部署环境的自动交付路径。

中期阶段的主要目标已经达到。下一阶段重点应放在最终证据固化、测试覆盖补强、目标微服务边界落地，以及部署回滚和数据安全能力完善上。